ISTE 330 Database Connectivity and Access

Esports Tournament Management System

Milestone 1: Specification, Design and Project Plan

Group 1

Member	RIT ID	Role
Dhruva Ravindran	dsr1378	Project Lead and Documentation
Abdullojon Ziyaev	za9369	Database Architect
Faizahnadir Nathershah	fn6379	Database Logic and Sample Data
Abdullah Naseem	afn5363	Java Back-End (JDBC)
Swati Poojary	sp6056	Java Front-End, Reports and Testing

Submitted: Friday, October 30, 2026

Contents

1. Problem Specification..... 3
2. Feature List..... 5
3. Database Design..... 6
4. Entity Relationship Diagram..... 9
5. UML Diagrams..... 10
6. Gantt Chart..... 13
7. Data Dictionary..... 14
8. Team Responsibilities..... 18
9. Risk Assessment..... 19

1. Problem Specification

1.1 The Problem

An esports organization needs software to manage its tournaments: the teams, coaches and players taking part, team registrations, match schedules, match results, standings and reports. Keeping this information in separate files makes it hard to keep accurate, and hard to answer simple questions such as who is leading a tournament or which players are on a team right now.

We will build a Java application that connects to a MySQL database through JDBC and keeps all of this information in one place. Each person logs in with their own account and can only use the functions that fit their role: Tournament Director, Coach or Player.

1.2 Who Uses the System

User type	Who they are	What they need to do
Tournament Director	Staff member who organizes and runs tournaments	Create and manage tournaments, schedule matches, record results, register teams, create accounts, see every report
Coach	Person responsible for one or more teams	Manage their team rosters, add or remove players, sign their teams up for tournaments, follow schedules and standings
Player	Competitor who belongs to one or more teams	See their own profile, teams, upcoming matches, results and personal statistics; update their own contact details

1.3 What the System Will Do

Functional requirements (FR):

- FR1 Log in: every user logs in with a username and password. The password is checked against a salted hash stored in the database; plain-text passwords are never stored.
- FR2 Role-based access: after login the user can only use the functions of the role stored on their account (Director, Coach or Player).
- FR3 Tournaments: a director can create, view, search, edit and delete tournaments (name, game, dates, location, prize pool, maximum teams, status).
- FR4 Teams and rosters: teams belong to one game and one coach. Players join and leave teams over time, and the history is kept.
- FR5 Registrations: a team can register for a tournament only if the tournament is open, the team plays the same game, it is not already registered, and the tournament is not full.
- FR6 Matches: a director schedules matches between two teams registered in the same tournament, then records the final score. Draws are not allowed.
- FR7 Player statistics: kills, deaths and assists are kept for each player in each completed match.
- FR8 Reports: at least five reports - tournament standings, match schedule, tournament registrations, team rosters and player statistics.
- FR9 Search: a director can search tournaments, teams and players by name.

1.4 Quality Requirements

- **Zero-confusion deployment:** the instructor runs one SQL script and one obvious Java entry point, then logs in with the test accounts. No searching through folders, code or configuration, and no extra data entry.
- **Security:** passwords are never stored as plain text; they are stored as salted hashes and checked against the database at login.
- **Safe SQL:** PreparedStatements for login, inserts, updates, deletes and searches; CallableStatements for stored procedures.

- **Data integrity:** primary keys, foreign keys and constraints enforce the business rules in the database itself.
- **Error handling:** database errors are caught and shown as clear messages instead of crashing the program.
- **Clean submission:** the final submission contains only the files that are required to run and evaluate the system.

1.5 Technical Requirements Covered

Course requirement	How we will meet it
Minimum 8 tables, PKs, FKs, normalization	11 tables in 3NF, every table has a primary key, 17 foreign key columns
At least two many-to-many relationships	Three: Player-Team (TeamMembership), Team-Tournament (TournamentRegistration), Player-Match (PlayerMatchStat)
Hashed passwords, login from database	UserAccount.password_hash + password_salt; Java hashes the typed password and compares
JDBC connections, ResultSets, CRUD, exceptions	DAO classes, one DatabaseConnection class, try-with-resources
PreparedStatements for login, insert, update, delete, search	All user-input SQL in the DAO layer uses PreparedStatement
Three stored procedures + CallableStatement	sp_RegisterTeamForTournament, sp_AddPlayerToTeam, sp_RecordMatchResult
Five meaningful reports	Standings, schedule, registrations, rosters, player statistics (built on SQL views)
One SQL script, test accounts, startup guide	One script (esports_tms_setup.sql) builds everything; Director, Coach and Player test accounts; one-page startup guide at Milestone 3

1.6 Deliverables and Deadlines

Milestone	Due	What we hand in
1	Fri 30 Oct 2026	This document: specification, feature list, ERD, UML, data dictionary, Gantt chart, team responsibilities, risk assessment
2	Wed 18 Nov 2026	Working demo: database and sample data, login with hashing, CRUD with PreparedStatements, one stored procedure through CallableStatement, two reports, updated UML
3	Wed 2 Dec 2026	Complete system, one SQL script, startup guide, README, test credentials, all reports and functions

1.7 Design Notes

These are the only business rules we have fixed so far. Anything else will be decided and documented as the project develops.

- A team plays one game, and a tournament is for one game. A coach can manage more than one team.
- The match winner is calculated from the scores rather than stored. Draws are not allowed.
- The matches table is named TournamentMatch because MATCH is a reserved word in MySQL.
- The team develops and tests on MySQL 9.7. The database script is delivered in Milestone 2, and required software versions will be listed in the README.

2. Feature List

Planned features, who uses them, which milestone delivers them, and the JDBC technique each one demonstrates.

ID	Feature	Users	Milestone	Technique shown
F01	Log in / log out	All	M2	PreparedStatement, SHA-256 hash check
F02	Role-based access (Director, Coach, Player)	All	M2	Role from UserAccount
F03	Create user account (hashed password)	Director	M2	PreparedStatement INSERT
F04	Create / edit / delete tournament	Director	M2	PreparedStatement INSERT, UPDATE, DELETE
F05	Search tournaments, teams, players	Director	M2	PreparedStatement with LIKE
F06	Register team for tournament	Director, Coach	M2	CallableStatement: sp_RegisterTeamForTournament
F07	Tournament standings report	All	M2	View vw_TournamentStandings, ResultSet
F08	Match schedule report	All	M2	View vw_MatchSchedule, ResultSet
F09	Schedule / edit / cancel match	Director	M3	PreparedStatement INSERT, UPDATE
F10	Record match result	Director	M3	CallableStatement: sp_RecordMatchResult
F11	Add player to team	Coach	M3	CallableStatement: sp_AddPlayerToTeam
F12	Remove player from team (keeps history)	Coach	M3	PreparedStatement UPDATE leave_date
F13	Edit team details	Coach	M3	PreparedStatement UPDATE
F14	Team roster report	Director, Coach	M3	View vw_TeamRoster
F15	Tournament registrations report	Director	M3	View vw_TournamentRegistrations
F16	Player statistics report	All	M3	View vw_PlayerStatistics
F17	View / update my profile	Player	M3	PreparedStatement SELECT, UPDATE
F18	View my teams, schedule and stats	Player	M3	PreparedStatement with player_id
F19	Clear error messages for database errors	All	M2-M3	SQLException handling

3. Database Design

3.1 Tables

The database esports_tms has 11 tables. The full ERD is in Section 4 and every column is listed in the data dictionary in Section 7.

Table	Purpose
UserAccount	One login account per person. Holds the role and the salted password hash used for authentication.
Director	Profile of a Tournament Director (1:1 with a DIRECTOR user account).
Coach	Profile of a Coach (1:1 with a COACH user account). A coach manages one or more teams.
Player	Profile of a Player (1:1 with a PLAYER user account).
Game	Video game titles that teams play and tournaments are held in.
Team	A competitive team for one game, managed by one coach.
TeamMembership	Junction table for the Player <-> Team many-to-many relationship, with membership history.
Tournament	A tournament for one game, run by one director.
TournamentRegistration	Junction table for the Team <-> Tournament many-to-many relationship.
TournamentMatch	A scheduled match between two registered teams in a tournament, with its result.
PlayerMatchStat	Junction table for the Player <-> TournamentMatch many-to-many relationship (per-match player statistics).

3.2 Many-to-Many Relationships

Each many-to-many relationship is implemented with its own junction table. No field ever holds a list of values.

Relationship	Junction table	Primary key	Foreign keys	Business meaning
Player <-> Team	TeamMembership	(team_id, player_id, join_date)	team_id -> Team player_id -> Player	A player can be on many teams over time; a team has many players. join_date in the key lets a player leave and later rejoin.
Team <-> Tournament	TournamentRegistration	(tournament_id, team_id)	tournament_id -> Tournament team_id -> Team	A team enters many tournaments; a tournament has many teams. A team can register only once per tournament.
Player <-> TournamentMatch	PlayerMatchStat	(match_id, player_id)	match_id -> TournamentMatch player_id -> Player	A player plays in many matches; a match has many players. Holds kills, deaths and assists.

Extra integrity rule: TournamentMatch uses composite foreign keys (tournament_id, home_team_id) and (tournament_id, away_team_id) that point to TournamentRegistration. The database itself therefore refuses a match between teams that are not registered in that tournament. A check constraint also stops a team from playing itself.

3.3 Normalization

Normal form	Rule	How our design meets it
1NF	Atomic values, no repeating groups	Every column holds one value. Team rosters, registrations and match stats are rows in junction tables, never comma-separated lists.

Normal form	Rule	How our design meets it
2NF	No partial dependency on part of a composite key	In the junction tables, every non-key column (e.g. team_role, seed_number, kills) depends on the whole key. Player names live in Player, not in TeamMembership.
3NF	No transitive dependency	Coach and game details are stored once and referenced by ID. The match winner and standings are calculated in views instead of being stored, so they can never disagree with the scores.

Login design: UserAccount holds only login data (username, hash, salt, role). Profile details are in Director, Coach or Player, linked one-to-one through a unique user_id. This keeps security data separate and avoids empty columns.

3.4 Stored Procedures (called with CallableStatement)

Procedure	Parameters	Checks before changing data
sp_RegisterTeamForTournament	IN tournament_id, IN team_id OUT success, OUT message	Tournament exists and is UPCOMING; team exists; team plays the tournament's game; not already registered; tournament not full
sp_AddPlayerToTeam	IN team_id, IN player_id, IN team_role OUT success, OUT message	Team and player exist; player not already a current member; roster below players_per_team + 2 substitutes
sp_RecordMatchResult	IN match_id, IN home_score, IN away_score OUT success, OUT message	Match exists and is SCHEDULED; scores 0-99; no draw. Sets status to COMPLETED

Every procedure returns a success flag and a plain-English message, so the Java program can show the user exactly why something was rejected. Each one will be tested on MySQL 9.7 with both valid and invalid inputs as part of Milestone 2.

3.5 Reports

#	Report	Source	Shows
1	Tournament standings	vw_TournamentStandings	Played, wins, losses, score difference and points (3 per win) per team
2	Match schedule	vw_MatchSchedule	Date, round, venue, teams, score, winner and status for each match
3	Tournament registrations	vw_TournamentRegistrations	Teams entered in each tournament with coach, date, seed and status
4	Team rosters	vw_TeamRoster	Current players of each team with role and join date
5	Player statistics	vw_PlayerStatistics	Matches played, total kills, deaths, assists and K/D ratio

3.6 Password Security

- Each account has its own random 32-character salt.
- Stored value = SHA-256(salt + password), saved as 64 hex characters.
- At login, Java reads the salt for the username, hashes the typed password the same way and compares. The plain password is never saved or printed.
- The SQL script will create the sample hashes with MySQL's SHA2() function. We checked that Java's MessageDigest produces exactly the same hash as SHA2() for the same salt and password.

3.7 Sample Data Plan

Data	Amount	Why
Tournaments	4 (1 completed, 1 ongoing, 2 upcoming)	Shows every status and lets the instructor test registration
Games	3	Shows that registration is limited to the matching game
Teams / coaches	8 teams / 4 coaches	Each coach manages two teams
Players	45	40 starters, 2 substitutes, 3 free agents to test Add Player
Matches	14 (10 completed, 4 scheduled)	Standings, schedule and Record Result can all be demonstrated
Player statistics	100 rows	Meaningful statistics report
Team history	2 players with past teams	Demonstrates the many-to-many relationship over time

4. Entity Relationship Diagram

Crow's foot notation. Two bars = exactly one. Circle + crow's foot = zero or many. Circle + bar = zero or one. Orange tables are junction tables. (U) = unique column.

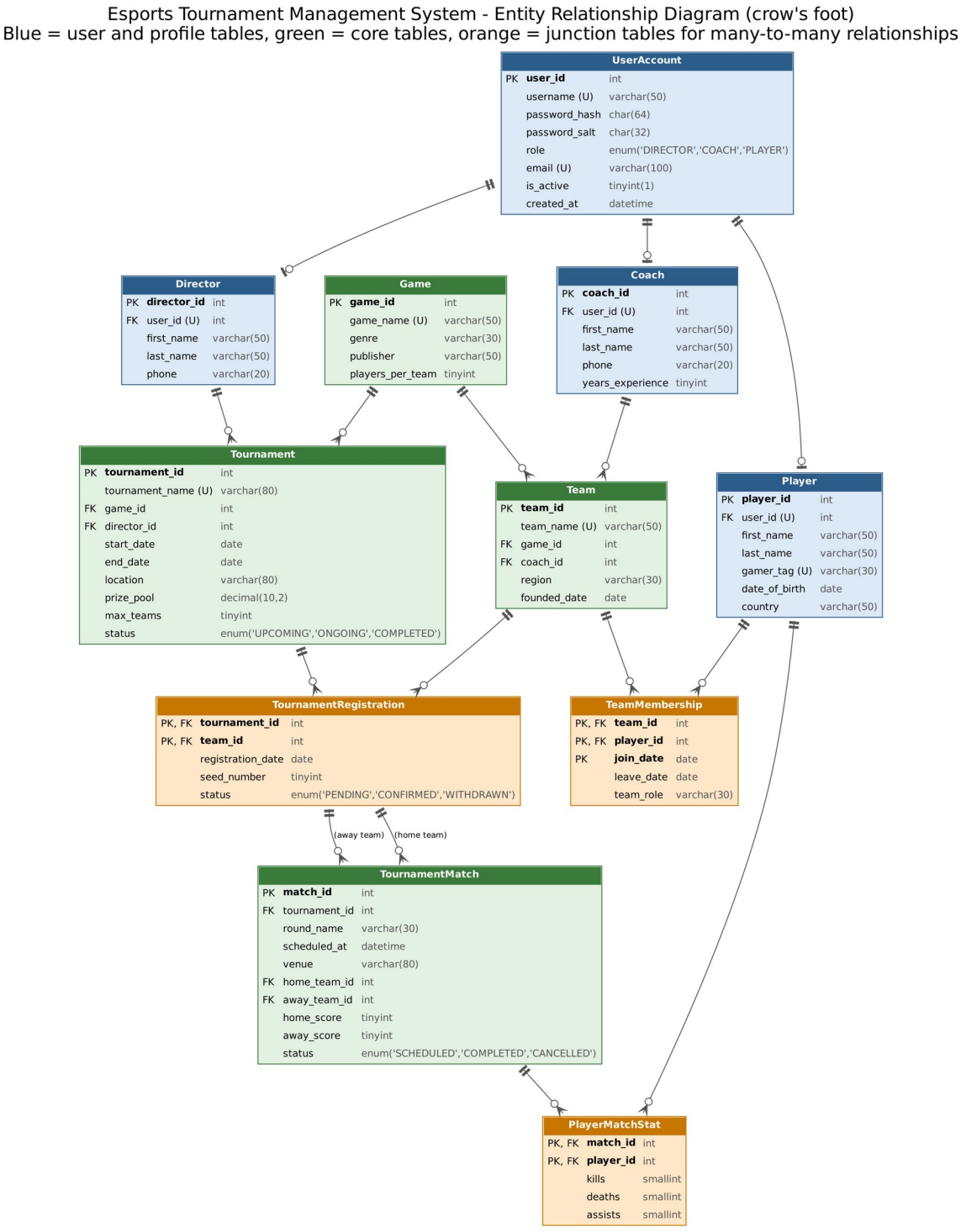


Figure 1. Entity Relationship Diagram (matches the planned database schema)

5. UML Diagrams

5.1 Use Case Diagram

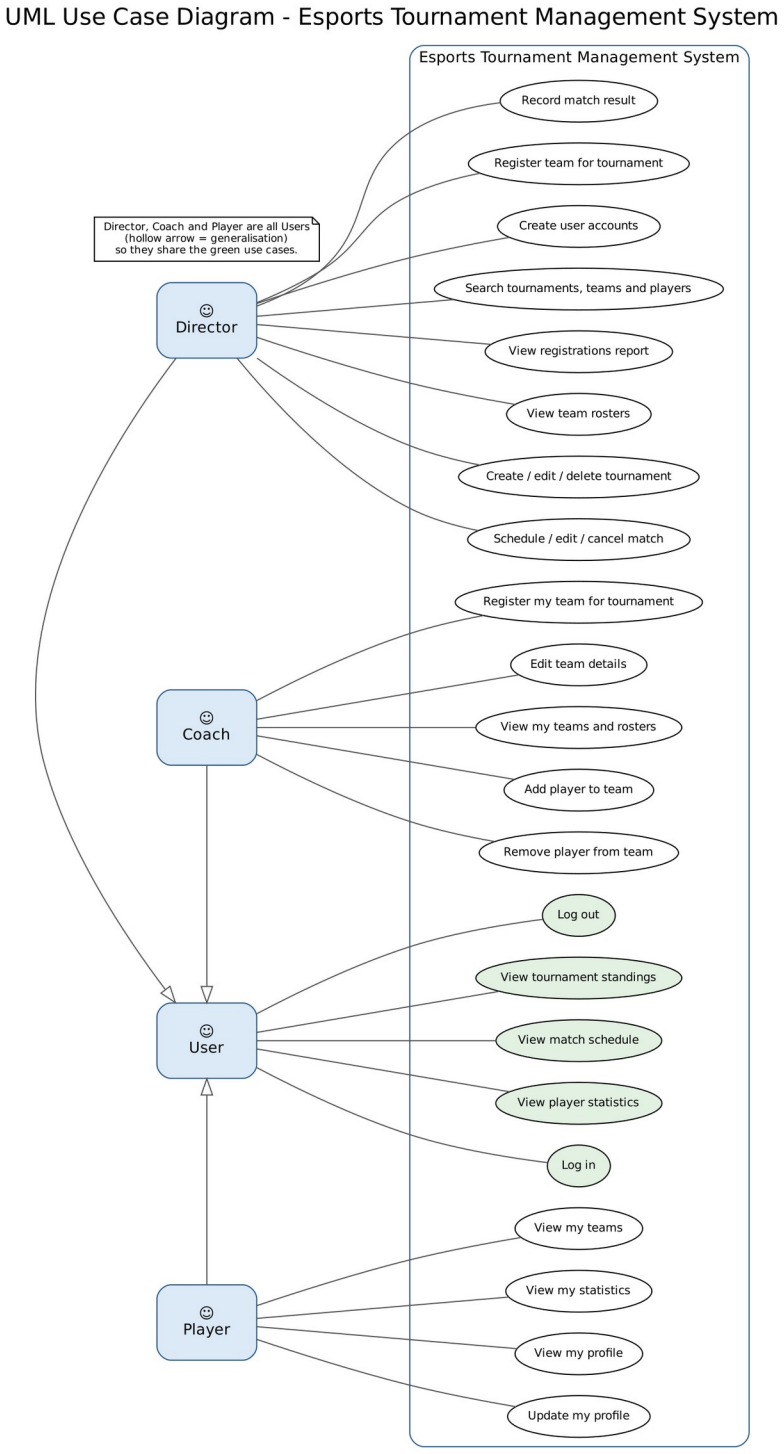


Figure 2. Use cases by user type

5.2 Class Diagram - Domain Model

One Java class per table in package model. Junction classes carry the extra data of each many-to-many relationship.

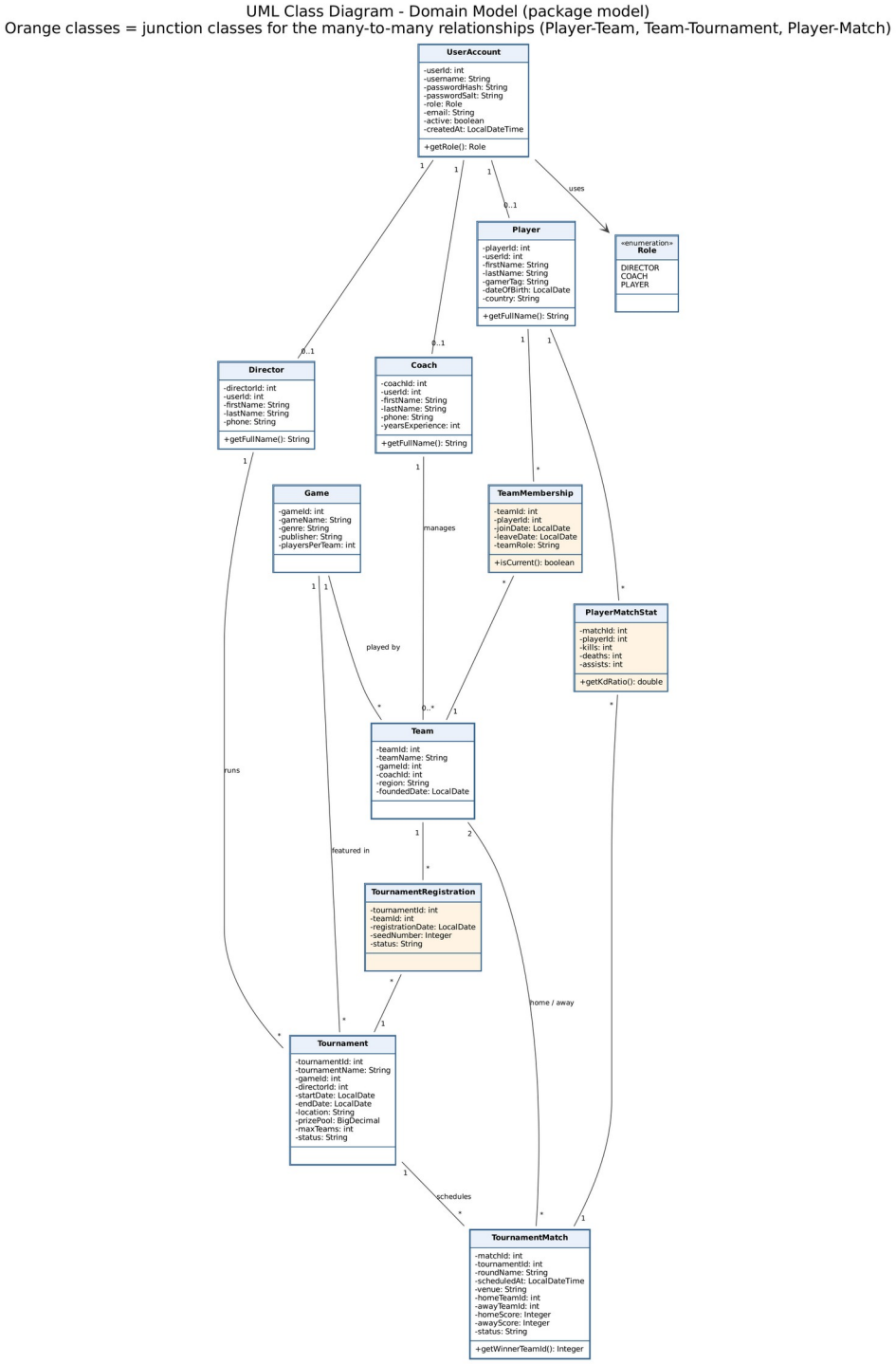


Figure 3. Domain model classes and multiplicities

5.3 Class Diagram - Application Structure

Initial design, to be updated at Milestone 2 as the spec requires. The UI never writes SQL. Menus call DAO classes; DAO classes are the only classes that use JDBC; all of them get connections from one DatabaseConnection class. Dashed arrows show dependencies.

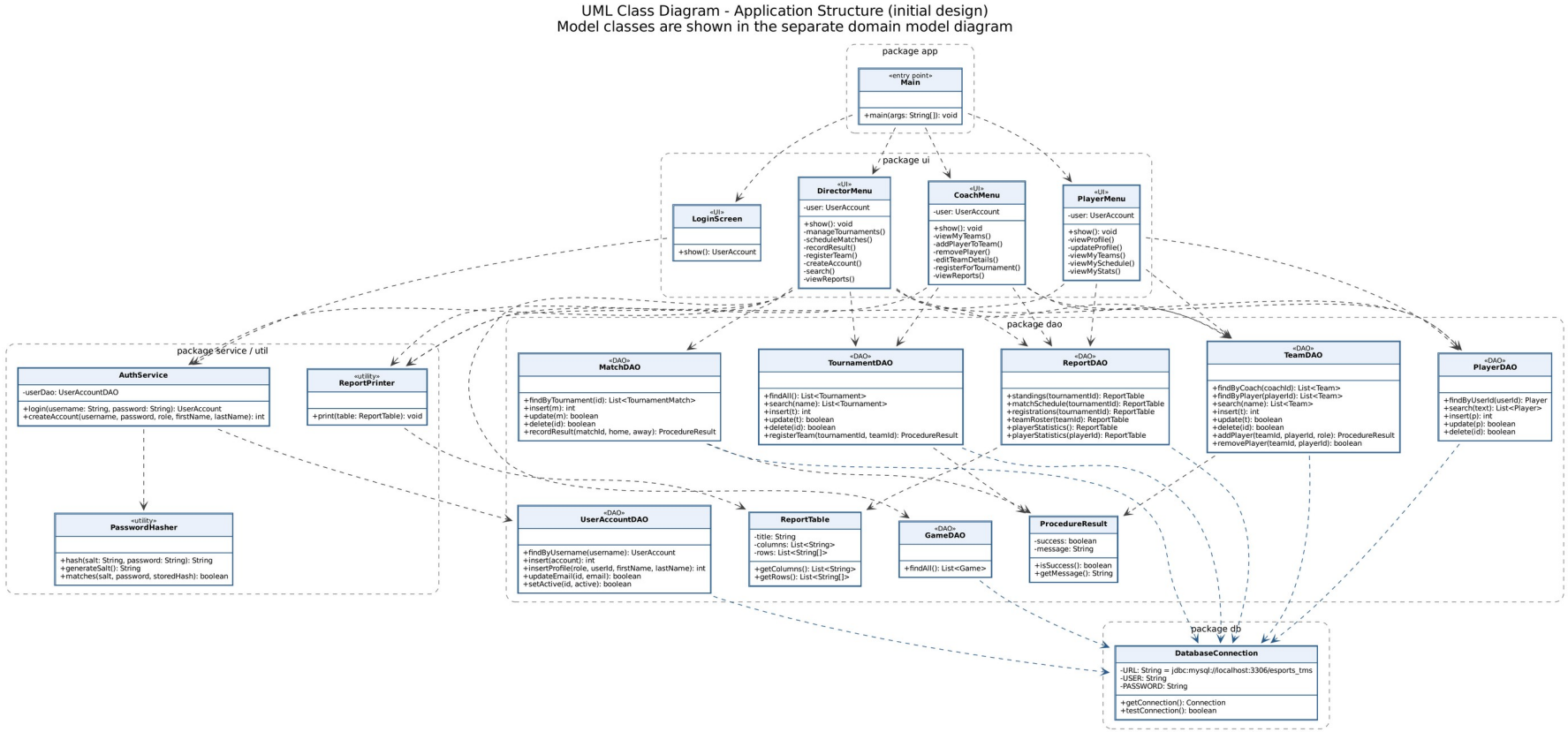


Figure 4. Initial Java package and class design

6. Gantt Chart

Built from our task plan. The editable GanttProject file is docs/milestone1/source/Project_Schedule.gan (open with ganttproject.biz). Each deadline has 1-2 days of buffer for clean-machine testing.

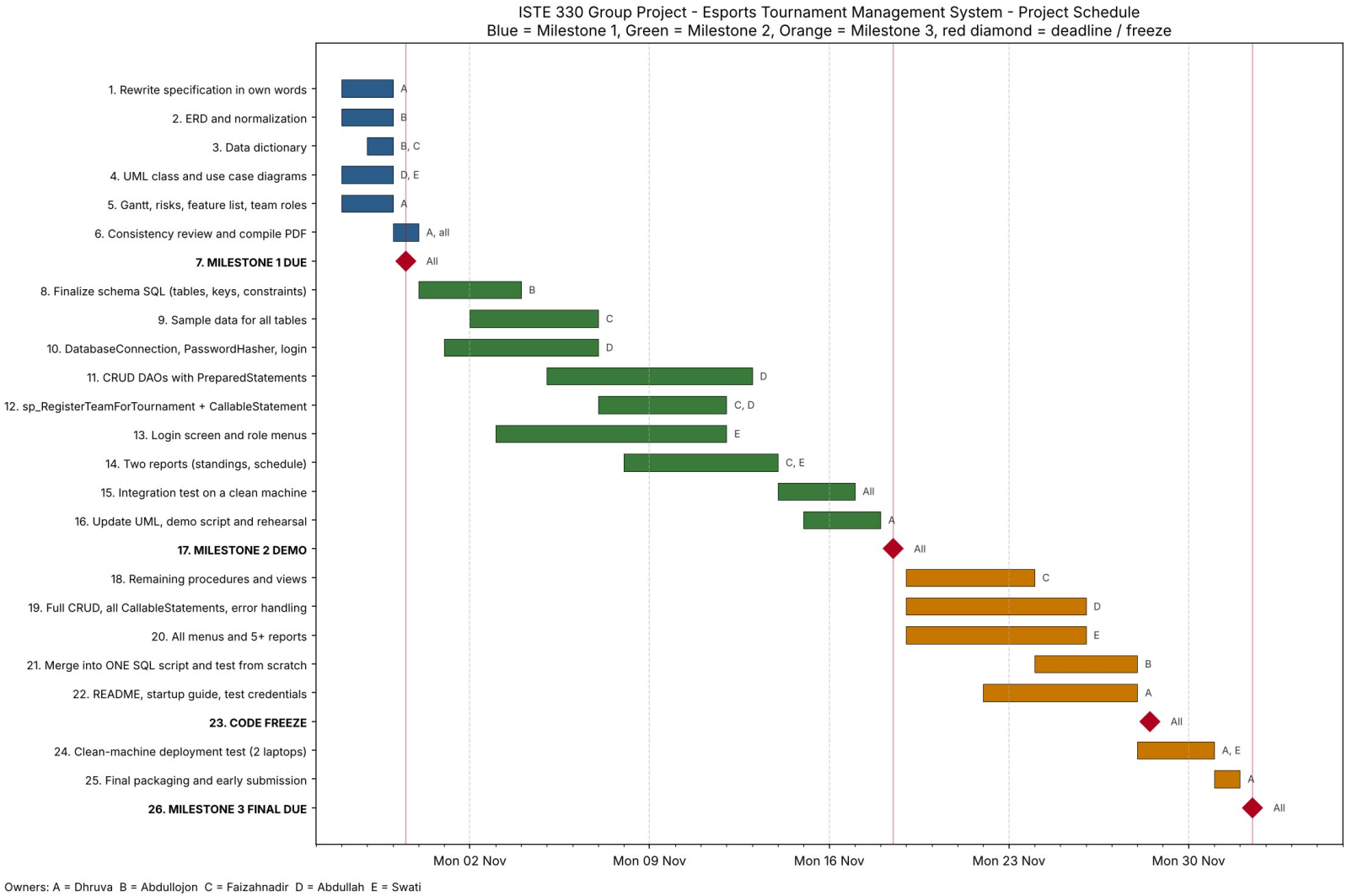


Figure 5. Project schedule, 28 Oct - 2 Dec 2026

7. Data Dictionary

Taken directly from the database design, so names and types will match the SQL script delivered in Milestone 2. Key: PK = primary key, FK = foreign key, UQ = unique.

7.1 UserAccount

One login account per person. Holds the role and the salted password hash used for authentication.

Column	Data type	Null	Key	References	Description
user_id	int AUTO_INCREMENT	No	PK		Unique account number (auto-generated).
username	varchar(50)	No	UQ		Login name; must be unique.
password_hash	char(64)	No			SHA-256 hash of salt + password, 64 hex characters. Plain passwords are never stored.
password_salt	char(32)	No			Random 32-character salt unique to this account.
role	enum('DIRECTOR','COACH','PLAYER')	No			User type: DIRECTOR, COACH or PLAYER. Controls the menu shown after login.
email	varchar(100)	No	UQ		Contact email; must be unique.
is_active	tinyint(1) default 1	No			1 = can log in, 0 = account disabled.
created_at	datetime default CURRENT_TIMESTAMP	No			Date and time the account was created.

7.2 Director

Profile of a Tournament Director (1:1 with a DIRECTOR user account).

Column	Data type	Null	Key	References	Description
director_id	int AUTO_INCREMENT	No	PK		Unique director number (auto-generated).
user_id	int	No	FK, UQ	UserAccount.user_id	Login account of this director (one account per director).
first_name	varchar(50)	No			Director's first name.
last_name	varchar(50)	No			Director's last name.
phone	varchar(20)	Yes			Contact phone number (optional).

7.3 Coach

Profile of a Coach (1:1 with a COACH user account). A coach manages one or more teams.

Column	Data type	Null	Key	References	Description
coach_id	int AUTO_INCREMENT	No	PK		Unique coach number (auto-generated).
user_id	int	No	FK, UQ	UserAccount.user_id	Login account of this coach (one account per coach).
first_name	varchar(50)	No			Coach's first name.
last_name	varchar(50)	No			Coach's last name.
phone	varchar(20)	Yes			Contact phone number (optional).

Column	Data type	Null	Key	References	Description
years_experience	tinyint default 0	No			Years of coaching experience (0 or more).

7.4 Player

Profile of a Player (1:1 with a PLAYER user account).

Column	Data type	Null	Key	References	Description
player_id	int AUTO_INCREMENT	No	PK		Unique player number (auto-generated).
user_id	int	No	FK, UQ	UserAccount.user_id	Login account of this player (one account per player).
first_name	varchar(50)	No			Player's first name.
last_name	varchar(50)	No			Player's last name.
gamer_tag	varchar(30)	No	UQ		In-game name; must be unique.
date_of_birth	date	No			Player's date of birth.
country	varchar(50)	No			Country the player represents.

7.5 Game

Video game titles that teams play and tournaments are held in.

Column	Data type	Null	Key	References	Description
game_id	int AUTO_INCREMENT	No	PK		Unique game number (auto-generated).
game_name	varchar(50)	No	UQ		Title of the game; must be unique.
genre	varchar(30)	No			Game genre, e.g. Tactical Shooter, MOBA.
publisher	varchar(50)	No			Company that publishes the game.
players_per_team	tinyint	No			Starting players per team (1-10). Roster limit = this + 2 substitutes.

7.6 Team

A competitive team for one game, managed by one coach.

Column	Data type	Null	Key	References	Description
team_id	int AUTO_INCREMENT	No	PK		Unique team number (auto-generated).
team_name	varchar(50)	No	UQ		Team name; must be unique.
game_id	int	No	FK	Game.game_id	Game this team competes in.
coach_id	int	No	FK	Coach.coach_id	Coach who manages this team.
region	varchar(30)	No			Home region of the team.
founded_date	date	No			Date the team was founded.

7.7 TeamMembership

Junction table for the Player <-> Team many-to-many relationship, with membership history.

Column	Data type	Null	Key	References	Description
team_id	int	No	PK, FK	Team.team_id	Team the player belongs to.
player_id	int	No	PK, FK	Player.player_id	Player on the team.
join_date	date	No	PK		Date the player joined. Part of the key so a player can rejoin later.
leave_date	date	Yes			Date the player left; NULL = currently on the team.
team_role	varchar(30) default Member	No			Role in the team, e.g. Captain, Member, Substitute.

7.8 Tournament

A tournament for one game, run by one director.

Column	Data type	Null	Key	References	Description
tournament_id	int AUTO_INCREMENT	No	PK		Unique tournament number (auto-generated).
tournament_name	varchar(80)	No	UQ		Tournament name; must be unique.
game_id	int	No	FK	Game.game_id	Game played in this tournament.
director_id	int	No	FK	Director.director_id	Director responsible for this tournament.
start_date	date	No			First day of the tournament.
end_date	date	No			Last day; must be on or after start_date.
location	varchar(80)	No			Venue or online server.
prize_pool	decimal(10,2) default 0.00	No			Total prize money (0 or more).
max_teams	tinyint	No			Maximum number of registered teams (2 or more).
status	enum('UPCOMING','ONGOING','COMPLETED') default UPCOMING	No			UPCOMING (open for registration), ONGOING or COMPLETED.

7.9 TournamentRegistration

Junction table for the Team <-> Tournament many-to-many relationship.

Column	Data type	Null	Key	References	Description
tournament_id	int	No	PK, FK	Tournament.tournament_id	Tournament the team entered.
team_id	int	No	PK, FK	Team.team_id	Team that registered.
registration_date	date	No			Date the team registered.
seed_number	tinyint	Yes			Seeding position (optional; set by the director).
status	enum('PENDING','CONFIRMED','WITHDRAWN') default CONFIRMED	No			PENDING, CONFIRMED or WITHDRAWN.

7.10 TournamentMatch

A scheduled match between two registered teams in a tournament, with its result.

Column	Data type	Null	Key	References	Description
match_id	int AUTO_INCREMENT	No	PK		Unique match number (auto-generated).
tournament_id	int	No	FK	TournamentRegistration.tournament_id, TournamentRegistration.tournament_id	Tournament this match belongs to.
round_name	varchar(30)	No			Stage of the tournament, e.g. Group Stage R1, Semi Final.
scheduled_at	datetime	No			Date and time the match is scheduled.
venue	varchar(80)	No			Where the match is played.
home_team_id	int	No	FK	TournamentRegistration.team_id	First team; must be registered in the same tournament.
away_team_id	int	No	FK	TournamentRegistration.team_id	Second team; must be registered in the same tournament and differ from home team.
home_score	tinyint	Yes			Final score of home team; NULL until completed.
away_score	tinyint	Yes			Final score of away team; NULL until completed. Draws not allowed.
status	enum('SCHEDULED','COMPLETED','CANCELLED') default SCHEDULED	No			SCHEDULED, COMPLETED or CANCELLED. Winner is calculated from scores, not stored.

7.11 PlayerMatchStat

Junction table for the Player <-> TournamentMatch many-to-many relationship (per-match player statistics).

Column	Data type	Null	Key	References	Description
match_id	int	No	PK, FK	TournamentMatch.match_id	Match the statistics are for.
player_id	int	No	PK, FK	Player.player_id	Player the statistics are for.
kills	smallint default 0	No			Eliminations by the player in the match (0 or more).
deaths	smallint default 0	No			Times the player was eliminated (0 or more).
assists	smallint default 0	No			Assists by the player in the match (0 or more).

8. Team Responsibilities

Each member owns one area from design to final submission, so there is always one clear person responsible for each part.

Member	RIT ID	Role	Owns
Dhruva Ravindran	dsr1378	Project Lead and Documentation	Specification, Gantt, risk register, feature list, README, startup guide, final packaging and submission
Abdullojon Ziyaev	za9369	Database Architect	ERD, normalization, data dictionary, tables and keys, owner of the single SQL setup script
Faizahnadir Nathershah	fn6379	Database Logic and Sample Data	Stored procedures, views, report queries, realistic sample data
Abdullah Naseem	afn5363	Java Back-End (JDBC)	DatabaseConnection, DAO classes, PreparedStatements, CallableStatements, password hashing, login, exception handling
Swati Poojary	sp6056	Java Front-End, Reports and Testing	Main entry point, login screen, Director/Coach/Player menus, report screens, end-to-end testing

8.1 Work by Milestone

Member	Milestone 1	Milestone 2	Milestone 3
Dhruva	Specification, feature list, Gantt, risks, compile document	Updated UML, demo script, rehearsal	README, startup guide, clean-machine test, final ZIP
Abdullojon	ERD, normalization, data dictionary	Final schema SQL with all keys and constraints	Merge everything into ONE SQL script and test it from scratch
Faizahnadir	Data dictionary descriptions, procedure and report design	Sample data, first stored procedure, two report queries	Remaining procedures, views, all report queries
Abdullah	Application class diagram	Connection, password hashing, login, CRUD, first CallableStatement	All DAOs, all CallableStatements, exception handling
Swati	Use case diagram, consistency review	Login screen, role menus, two report screens	All menus and reports, end-to-end testing

8.2 How We Work

- All work is kept in one shared GitHub repository, with a branch for each milestone.
- The Database Architect owns the SQL script, so there is only ever one version of the database.
- Changes to tables or columns are agreed by the group first, because they affect the Java code.
- Regular check-ins; anyone who falls behind tells the group early.
- A feature counts as done only when it runs on a second member's computer.

9. Risk Assessment

#	Risk	Likelihood	Impact	Mitigation	Owner
1	Program does not start on the instructor's computer (automatic F for Milestone 3)	Medium	Very high	One SQL script and one obvious Java entry point; full test on a computer that did not build the code before every submission	Dhruva, Swati
2	Wrong MySQL user/password stops the connection	High	High	Connection settings kept in one place and explained in the README and startup guide	Abdullah
3	Java and SQL hash passwords differently, so nobody can log in	Medium	High	Agreed SHA-256(salt + password); checked that Java's hash matches MySQL's SHA2() output	Abdullah, Faizahnadir
4	Schema changes late and breaks Java code	Medium	High	Schema agreed early in Milestone 2; later changes agreed by the group; data dictionary kept in line with the real database	Abdullojon
5	Merge conflicts or lost work in Git	Medium	Medium	Small commits, pull before push, one owner per file area, one owner for the SQL script	All
6	A member falls behind or drops the course	Medium	High	Regular check-ins; every area has a backup member; all work is in the shared repository	Dhruva
7	MySQL or Java version differences between team computers and the instructor's	Low	Medium	Team standard is MySQL 9.7; list versions in the README; test the script on a fresh MySQL 9.7 database before each submission	Abdullojon
8	Submission contains extra or missing files	Medium	High	Checklist of required files; unzip and run the final submission before uploading	Dhruva
9	Sample data too thin to show features	Low	Medium	Data plan in Section 3.7 covers every status, report and procedure path	Faizahnadir